

Project Report — Health and Fitness Club Management System

Course: EGEN5208W — Database Management Systems **Term:** Winter 2026

1. Introduction

This report documents the design and implementation of a Health and Fitness Club Management System. The system enables members to register, track health metrics and fitness goals; trainers to manage availability and view schedules; and administrators to manage room bookings and equipment maintenance.

The system is backed by PostgreSQL and accessed through a Python Flask web application with a React (Vite) frontend and Tailwind CSS for the user interface.

2. ER Model

The conceptual ER model uses UML-like notation and contains **13 entities** and **13 relationships**. The full diagram is provided in `docs/ER_Diagram.pdf`.

2.1 Entities

#	Entity	Primary Key	Description
1	Member	member_id	Registered club members
2	Trainer	trainer_id	Staff who conduct sessions and classes
3	Admin	admin_id	Administrative staff
4	FitnessGoal	goal_id	Goals set by members (e.g., target weight)
5	HealthMetric	metric_id	Append-only health measurements
6	TrainerAvailability	availability_id	Time slots when trainers are available
7	Room	room_id	Physical rooms in the club
8	Equipment	equipment_id	Gym equipment tracked for maintenance
9	PersonalSession	session_id	One-on-one training sessions
10	GroupClass	class_id	Group fitness classes
11	ClassEnrollment	enrollment_id	Junction entity: Member ↔ GroupClass (M:N)
12	EquipmentMaintenance	log_id	Maintenance issue logs per equipment
13	Payment	payment_id	Simulated billing records

2.2 Relationships

#	Relationship	Cardinality	Description
---	--------------	-------------	-------------

1	Member → FitnessGoal	1:N	A member sets many goals
2	Member → HealthMetric	1:N	A member records many metrics
3	Member → PersonalSession	1:N	A member books many sessions
4	Member → ClassEnrollment	1:N	A member enrolls in many classes
5	Trainer → TrainerAvailability	1:N	A trainer defines many slots
6	Trainer → PersonalSession	1:N	A trainer conducts many sessions
7	Trainer → GroupClass	1:N	A trainer teaches many classes
8	Room → PersonalSession	1:N	A room hosts many sessions
9	Room → GroupClass	1:N	A room hosts many classes
10	Room → Equipment	1:N	A room contains many equipment items
11	GroupClass → ClassEnrollment	1:N	A class has many enrollments
12	Equipment → EquipmentMaintenance	1:N	Equipment has many maintenance logs
13	Member → Payment	1:N	A member makes many payments

Many-to-many: Member ↔ GroupClass is resolved through the ClassEnrollment junction entity.

2.3 Participation Constraints

- Every PersonalSession requires exactly one Member, Trainer, and Room (total participation)
- Every GroupClass requires exactly one Trainer and Room (total participation)
- Every ClassEnrollment references exactly one GroupClass and one Member (total participation)
- Members, Trainers, and Rooms may have zero related sessions/classes (partial participation)

3. Mapping ER Model to Relational Schema

The full relational schema diagram is provided in `docs/Relational_Schema.pdf`.

Each entity maps directly to a PostgreSQL table. The mapping follows standard rules:

ER Concept	Relational Mapping
Entity	Table
Entity attributes	Table columns
Primary key	SERIAL surrogate key (auto-increment)
1:N relationship	Foreign key in the "many" side table
M:N relationship (Member ↔ GroupClass)	Junction table <code>class_enrollment</code> with FKs to both <code>member</code> and <code>group_class</code> , plus a <code>UNIQUE(class_id, member_id)</code> constraint
Unique email per role	UNIQUE constraint on email in <code>member</code> , <code>trainer</code> , and <code>admin</code>

Key Mapping Details

- **Separate user tables:** Member, Trainer, and Admin are separate tables (not a single user table) because each role has distinct attributes. See Section 4.1 for justification.
 - **ClassEnrollment:** Resolves the M:N between Member and GroupClass. Uses a surrogate `enrollment_id` as PK with a composite unique constraint on `(class_id, member_id)` to prevent duplicate enrollments.
 - **Equipment → Room:** Equipment references Room via `room_id` FK, satisfying the requirement that equipment is associated with a location.
 - **All FK constraints** use `REFERENCES ... ON DELETE CASCADE` or default restrict behavior as appropriate.
-

4. Design Decisions

4.1 Separate User Tables

Three separate tables (`member` , `trainer` , `admin`) were used instead of a single `user` table with a role column. This decision was made because:

- Each role has distinct attributes (e.g., trainers have `specialization` , members have `dob` and `gender`)
- It avoids NULL columns that would be irrelevant to certain roles
- Foreign key references are clearer (e.g., `personal_session.trainer_id` references `trainer` , not a generic user table)

4.2 Append-Only Health Metrics

The `health_metric` table is designed as append-only. Members insert new records but never update or delete existing ones. This preserves a complete history of measurements and supports trend analysis on the dashboard.

4.3 Equipment Maintenance Separation

Equipment maintenance logs are stored in a separate `equipment_maintenance` table rather than as a status history column. This allows:

- Multiple maintenance issues per equipment item
- Tracking of issue lifecycle (reported → in progress → resolved)
- The `equipment.status` field reflects current operational status independently of historical maintenance records

4.4 Room Booking via Trigger

Room double-booking prevention is implemented as a PostgreSQL trigger (`fn_prevent_room_double_booking`) rather than application-level validation. This ensures data integrity regardless of how data is inserted — whether through the application, direct SQL, or future integrations.

4.5 Dashboard View

A PostgreSQL view (`member_dashboard_view`) aggregates data from multiple tables using LATERAL joins to power the member dashboard. This keeps complex aggregation logic in the database layer where it performs best.

5. Normalization to Third Normal Form (3NF)

5.1 First Normal Form (1NF)

All tables satisfy 1NF:

- Every attribute contains only atomic (indivisible) values
- No repeating groups or arrays
- Each table has a defined primary key

5.2 Second Normal Form (2NF)

All tables satisfy 2NF:

- All non-key attributes are fully functionally dependent on the entire primary key
- Since all tables use single-column surrogate primary keys (SERIAL), partial dependencies are impossible
- The composite candidate key (class_id, member_id) in class_enrollment is enforced via a UNIQUE constraint; the surrogate enrollment_id is the primary key

5.3 Third Normal Form (3NF)

All tables satisfy 3NF — no transitive dependencies exist:

Table	Non-Key Attributes	Justification
member	name, email, dob, gender, phone, password_hash, created_at	All depend directly on member_id
trainer	name, email, phone, specialization, password_hash	All depend directly on trainer_id
admin	name, email, password_hash	All depend directly on admin_id
fitness_goal	member_id, goal_type, target_value, start_date, end_date, status	All depend on goal_id; member_id is a FK, not a transitive dependency
health_metric	member_id, weight, body_fat_pct, blood_pressure, heart_rate, recorded_at	All depend on metric_id
trainer_availability	trainer_id, available_date, start_time, end_time	All depend on availability_id
room	room_name, capacity	All depend on room_id
equipment	name, type, room_id, status, purchase_date	All depend on equipment_id; room_id is a FK; status is the current status, not derived from maintenance logs
personal_session	member_id, trainer_id, room_id, session_date, start_time, end_time, status	All depend on session_id

group_class	class_name, trainer_id, room_id, class_date, start_time, end_time, max_participants	All depend on class_id
class_enrollment	class_id, member_id, enrolled_at	All depend on enrollment_id
equipment_maintenance	equipment_id, issue_description, reported_date, resolved_date, status	All depend on log_id
payment	member_id, amount, payment_status, payment_date, payment_method	All depend on payment_id; member_id is a FK, not a transitive dependency

5.4 No Derived Attributes

No computed or derived attributes are stored:

- **Age** is computed from `dob` at query time, not stored
- **Total classes attended** is computed via `COUNT(*)` on `class_enrollment` joined with `group_class` where `class_date < CURRENT_DATE`
- **Upcoming sessions count** is computed via query on `personal_session`

Health metric design: The `health_metric` table uses a wide-table design (columns for weight, body_fat_pct, blood_pressure, heart_rate) rather than an EAV pattern (metric_type + metric_value). This was chosen because: (1) the set of metric types is fixed and known; (2) each record naturally captures multiple measurements at one timestamp; (3) it simplifies queries for dashboard aggregation and avoids repeated self-joins.

6. Advanced SQL Features

6.1 View

View	Purpose
member_dashboard_view	Aggregates data from <code>member</code> , <code>health_metric</code> , <code>fitness_goal</code> , <code>class_enrollment</code> , and <code>personal_session</code> using LATERAL joins to provide latest health metrics, active goal count, classes attended, and upcoming sessions per member

6.2 Triggers (7 functions, 10 triggers)

Trigger Function	Tables	Purpose
fn_prevent_room_double_booking	personal_session, group_class	Prevents time overlaps in the same room across both tables
fn_prevent_member_overlapping_sessions	personal_session	Prevents a member from being double-booked
fn_prevent_trainer_double_booking	personal_session, group_class	Prevents a trainer from being booked in overlapping time slots

fn_prevent_full_class_enrollment	class_enrollment	Rejects enrollment when a class reaches max_participants
fn_health_metric_immutable	health_metric	Blocks UPDATE and DELETE to enforce append-only history
fn_prevent_trainer_availability_overlap	trainer_availability	Prevents overlapping availability slots for the same trainer
fn_verify_trainer_availability	personal_session	Ensures the trainer has an availability slot covering the session time

6.3 Stored Functions

Function	Purpose
fn_check_booking_conflicts(member, trainer, room, date, start, end)	Returns a set of conflict rows (member/trainer/room) for a proposed booking, used by the application to show detailed conflict messages
fn_trainer_slot_booking_status(trainer_id, member_id)	Returns availability slots enriched with existing session/class bookings, used to display the booking calendar

6.4 Indexes (9)

Index	Columns	Purpose
idx_health_metric_member_recorded	(member_id, recorded_at DESC)	Optimizes health history queries sorted by date
idx_personal_session_trainer	(trainer_id)	Speeds up trainer schedule lookups
idx_personal_session_member	(member_id)	Speeds up member session queries
idx_group_class_trainer	(trainer_id)	Speeds up trainer class queries
idx_group_class_room	(room_id)	Speeds up room booking conflict checks
idx_class_enrollment_class	(class_id)	Speeds up enrollment count queries
idx_class_enrollment_member	(member_id)	Speeds up member enrollment lookups
idx_payment_member	(member_id)	Speeds up member payment history
idx_trainer_availability	(trainer_id, available_date)	Speeds up availability lookups by trainer and date

7. Implementation

7.1 Architecture

The application follows a standard Flask blueprint architecture:

- `backend/__init__.py` — application factory
- `backend/db.py` — database connection management using `psycopg2`
- `backend/routes/auth.py` — authentication, login, registration, role decorators
- `backend/routes/member.py` — member operations (1-4)
- `backend/routes/trainer.py` — trainer operations (5-6)
- `backend/routes/admin.py` — admin operations (7-8)

7.2 Role-Based Access Control

- Authentication uses Flask sessions with werkzeug's `pbkdf2:sha256` password hashing
- A `role_required(role)` decorator checks both authentication and role authorization
- The `load_logged_in_user` function runs before every request to load the current user from the database

7.3 Error Handling

- Centralized error codes and message templates in `backend/errors.py` (e.g., `AUTH_001`, `VAL_006`, `BOOK_003`)
- `make_error(code, **params)` returns structured JSON responses with `error`, `error_code`, and HTTP status
- `parse_db_error()` maps PostgreSQL trigger/constraint messages to user-friendly error codes
- All database operations are wrapped in try/except blocks with commit on success, rollback on failure
- The frontend displays error messages from the JSON response using toast notifications

7.4 SQL Interaction

All database interactions use parameterized SQL queries via `psycopg2` (no ORM). This satisfies the requirement for direct SQL usage and prevents SQL injection.

8. Operations Summary

#	Operation	SQL Statements Used
1	User Registration	INSERT INTO member
2	Profile Management	UPDATE member; INSERT INTO fitness_goal; UPDATE fitness_goal; INSERT INTO health_metric
3	Health History	SELECT FROM health_metric ORDER BY recorded_at DESC
4	Dashboard	SELECT FROM member_dashboard_view; SELECT FROM fitness_goal; SELECT joins across personal_session, group_class, class_enrollment
5	Set Availability	INSERT INTO trainer_availability; SELECT for overlap check; DELETE
6	Schedule View	SELECT joins: personal_session + member + room; group_class + room + class_enrollment; LATERAL join for member health data
7	Room Booking	INSERT INTO personal_session; INSERT INTO group_class (trigger enforces no overlap); SELECT union of sessions and classes

8	Equipment Maintenance	INSERT INTO equipment_maintenance; UPDATE equipment; UPDATE equipment_maintenance; SELECT with status filter
---	-----------------------	--

9. Testing and CI/CD

9.1 Backend Tests

- **225 tests** using pytest with 100% code coverage (enforced via `pytest-cov --cov-fail-under=100`)
- Tests cover all route handlers, error paths, validation branches, and utility functions
- Database calls are mocked with `unittest.mock` so tests run without a live PostgreSQL instance

9.2 Frontend Tests

- **54 test suites** (949 tests) using Vitest with jsdom environment
- 100% coverage across statements, branches, functions, and lines via `@vitest/coverage-istanbul`
- ESLint with `eslint-plugin-react-hooks` and `eslint-plugin-react-refresh` enforces code quality with zero warnings

9.3 CI Pipeline

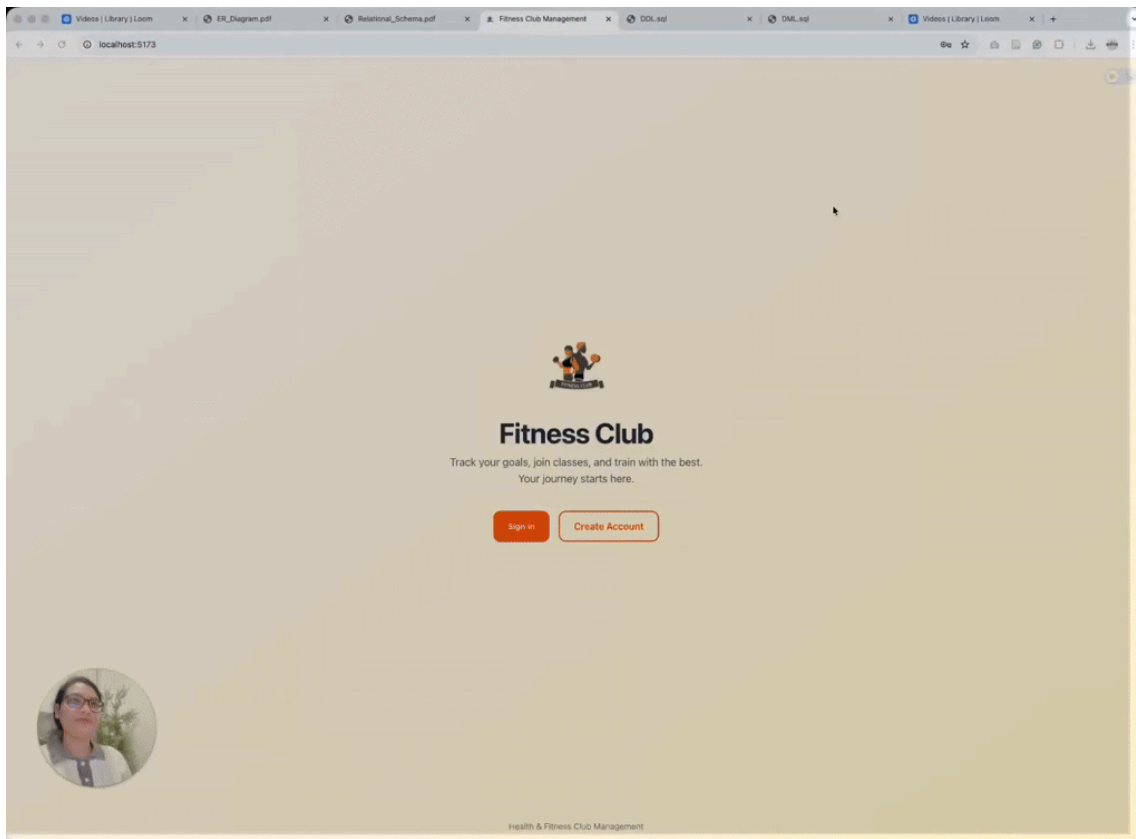
A GitHub Actions workflow runs on every push and pull request to `develop` :

- **Backend job:** installs dependencies, runs `pytest --cov=backend --cov-fail-under=100`
- **Frontend job:** installs dependencies, runs `eslint .` (lint), then `vitest run --coverage` (tests with coverage thresholds)

10. Challenges

- **Trigger conflict messages:** PostgreSQL trigger `RAISE EXCEPTION` messages are opaque strings. We wrote `parse_db_error()` with regex patterns to extract structured information (dates, times, conflict types) from these messages and convert them into user-friendly error responses.
- **Cross-table room booking:** Room double-booking prevention must check both `personal_session` and `group_class` tables simultaneously. A single trigger function queries both tables to detect overlaps, which was more complex than a simple unique constraint.
- **Booking conflict UX:** When a booking fails, users need to know *why* (member busy, trainer busy, or room busy) and *when* (the conflicting time slot). The `fn_check_booking_conflicts` stored function returns multiple conflict rows, which the application aggregates into a single detailed error message.
- **Append-only health metrics:** Enforcing immutability at the database level (via triggers that block UPDATE and DELETE) required careful coordination with the application layer, which must handle the resulting errors gracefully.

11. Video Demonstration



12. Conclusion

The system meets all specified requirements: 13 entities in 3NF, 13 relationships, 8 fully functional operations, role-based access control, database-level constraint enforcement, and a clean web interface. All SQL is parameterized and executed directly against PostgreSQL without an ORM.